ECE 3218 Lab #6 - The Timer Unit

Before the Lab

The AVR microcontroller, like many others, has a sophisticated timer system. This system can be used to generate an interrupt when a certain amount of time elapses. It can also generate a pulse or a square wave with a specified duration. The timer can also be used to measure the duration of a pulse or to count external pulses.

In this lab we will use the Timer1 unit, which supports 16-bit timer values (Timer 0 and 2 are for 8-bit operations only).

The timer is based on a free running 16-bit counter with independent channels that can be used either for input capture or for output compare functions. The input capture functions automatically record the counter's value when a selected transition is detected on the timer's input pin. The output compare functions can generate periodic output signals or interrupts. The interrupts can be used, through the ISR, to execute specific code periodically. When used that way, the timer interrupts are sometimes called a heartbeat.

The input capture function has a dedicated 16-bit register, `ICR1`, to which the value of the main timer register, `TCNT1`, is written when a triggering event occurs. Output compare has two 16-bit registers, `OCR1A` and `OCR1B`, in which values can be stored. These are continuously compared to the `TCNT1`. When one of the compare registers matches the timer register, events are triggered depending on how the output compare function is set up. All the input capture and output compare functions have individual interrupt controls and interrupt vectors. Refer to the Atmega328P data sheet for a description of the interrupt vector table. Control bits in various registers permit software to control the edge(s) that trigger input capture functions and the automatic actions that result from output compare functions.

To initialize a timer/counter setup the `TCCR1{A, B, C}` registers. These registers include bits that enable the timer and configure it. One particularly useful feature is the 'prescaler' (in TCCR1B) that causes the timer to run at speeds slower than the system clock.

To use output compare function A:

1. Program `TCCR1B` bits `CS1[2:0]` to select the clock source (possibly prescaled) to drive the timer.
2. Program `TCCR1A` bits `COM1A1:0` to select the effect the compare action will have on the output pin `OC1A` (toggle, rise, fall, nothing).
3. Program PB1 to output mode in `DDRB` to drive pin `PB1` pin with the `OC1A` signal.

To use interrupts, enable the OCIE1A interrupt in TIMSK1 (Timer 1 Interrupt Mask register), and enable the global interrupt flag (SEI). To use the timer without interrupts, poll the OCF1A bit in the TIFR1 register – if the bit is set, a compare match has occurred. This bit is automatically cleared when an associated ISR is executed. It can be cleared also by programmatically writing a logic one into its location, which must be done if interrupts are not being used.

To use the input capture function:

1. Program TCCR1B bits CS1[2:0] to select the clock source that will drive the timer, bit ICNC1 to enable the input capture noise canceller, and bit ICES1 to determine the edge detection logic (0 -> capture on falling edge, 1 -> capture on rising edge).
2. Program PB0 to input mode using DDRB to let the ICP1 (input capture signal) be received through pin PB0.
3. For interrupts, enable the ICIE1A interrupt in TIMSK1 (Timer 1 Interrupt Mask register), and enable the global interrupt flag (SEI). To use the input capture function without interrupts, poll the ICF1A bit in the TIFR1 register – if the bit is set, the time capture has happened. This bit is automatically cleared when the interrupt vector (ISR) is executed. It can be cleared also by programmatically writing a logic one into its location, which must be done if interrupts are not being used.

More information on the timer is available in the Atmega328P data sheet (Atmel doc8271). The timer has subtleties that can trip you up if you are not thoroughly familiar with its operation.

For easier connection, bring out the PBO (ICP1, input to the Input Capture function) the PB1 (OC1A, output from the Output Compare 1 A function) and a ground to a breadboard.

Programs:

1. Write a subroutine that uses interrupts to generate a rectangular wave with duty cycle, dc, prescale value, sp, and period, pw, in samples on pin PB1 (OC1A). That is, passed pw , sp, and d, the subroutine computes the widths of the high and the low portions of the waveform and sets up for an output compare interrupt on Timer1.

   The duty cycles that will be used are 12.5%, 25%, 50%, 75%, and 87.5%. The first three correspond to factors 0.125 = 2-3, 0.25 = 2-2, and 0.5 = 2-1. The width of the high pulse in the first case is 0.125pw. But that is equal to pw right shifted 3 times. Similarly, the second duty cycle has a high pulse width equal to pw right shifted twice, and the third is pw right shifted once. Note also that the 75% duty cycle is the logical NOT of the 25% duty cycle wave and that the 87.5% duty cycle is NOT (12.5% duty cycle wave). The subroutine can be written to expect only these five cases. (But note that waveform period pw could, in theory, be anything between 2 and 32767 samples.) When called, the subroutine should compute and save the high period and the low period in two 2-byte memory locations where they can be retrieved by the OCA1 ISR.

In the subroutine, before enabling interrupts globally, set DDRB pin 1 for output and write a 1 to that pin in the port register (PORTB pin 1). Put the width in samples for the logic high period in OCR1A. Using TCCR1A, set up the timer to toggle the OC1A pin on compare match. Also, clear bits WGM11 and WGM10 in TCCR1A. Enable an interrupt on output-compare-match-with-register-A by setting the correct pin in the TIMSK1 register. Set up the pins for TCCR1B in a general purpose register. That will include the bit for the clear timer on compare match (CTC) mode and the prescale value (see below) for the timer. Do not output the GP register to TCCR1B until just before enabling interrupts globally. That is because setting the prescale value starts the timer. (That also means you can stop the timer by clearing the prescaler bits in TCCR1B.) After setting up Timer1 and enabling interrupts, it returns to the main program (unspecified at this point).

The first time the ISR is called it will be because the duration of the logic high was reached. The OC1A pin will have been toggled and the timer register will have been cleared. The timer is still running. Write the time (in counter clicks) for the logic low output to OCR1A and return. The next time the ISR is called, write the time for the logic high period to OCR1A and return. Each time the ISR is called and performs a high to low toggle, write OCR1A with the logic low period and each time it's called and does a low to high toggle write OCR1A with the logic high period. Of course, that means you will be swapping one period for the other on successive calls of the ISR. The count value (high + low) that corresponds to the fundamental frequency of the pulse wave is a function of the prescaler value you select.

Use fw = 440 Hz.

To determine the prescaler value answer the following question:

a) What is the frequency, fcc, of the system clock?
b) What are the prescaler values available to Timer1?
c) If the prescaler value is sp, what is the timer/counter frequency, ftc?
d) If the frequency of the waveform is fw Hz, then what is its period in seconds, pw?
e) Given: variables ftc and pw, that the counter starts at zero, and that the counter has enough bits to hold the result, find an expression for the count after pw seconds.
f) Find the expression for the count in terms of fcc, sp, and pw.
g) What is cmax, the maximum value that Timer1 can hold?
h) Select sp as the smallest of the available prescaler values such that after pw seconds the count in T/C 1 is less than cmax + 1. What is sp?
i) Given the sp, that you chose, what is the count after pw seconds (starting from 0).

2. Write a program that calls the subroutine from problem 1 to play a sequence of at least 10 500 ms duration tones, each with a different period, pw, but a constant 50% duty cycle. This program successively reads periods, pw, from a list of 2-byte numbers in RAM and calls the subroutine. The main program's flow is to read the period, call the subroutine, wait 500 ms, stop the clock, wait another 500 ms, then repeat until all the tones are played. The 500 ms timer can be software-based like the timers you have used previously, or you can use

Timer0 or Timer2, if you prefer. In the lab, you will connect a piezo speaker to PB1 and listen to the “music” you make.

3. Write a program that will measure the period of a signal from a function generator and generate an output signal with half the frequency of the input. Use Input Capture 1 (IC1) to measure the period; set up an ISR that will be called whenever a leading (or trailing) edge is detected. Each time the ISR is called it reads the content of the input capture register and the saved value of the register from the previous call. By computing the difference between the current count and the previous, the period of the pulse can be estimated. Note that the timer may have rolled over between counts. Include logic to detect that situation and nevertheless, to compute the correct value. Store the 16-bit value in a variable (a specific, labeled memory location). That value will be used as half the period of the output wave to be generated via output compare.

   Note that Timer1 can be used simultaneously for input capture and output compare. Here you want to use the input capture to measure the pulse width coming from the signal generator and use output compare to generate a square wave that can be displayed on a scope. There will be separate ISRs for each. There is, however, only one Timer1 count register. That implies that you should not use the CTC mode. Instead let the timer run free and roll over after hitting its maximum. You will still want to use the toggle on output compare mode.

   During each input capture ISR, compute the pulse width assuming the counter will not roll over more than once between computations. (Make sure you have the signal generator’s period set and the timer’s prescaler set so that there will never be a multiple roll-over.)

   During each output compare ISR, read the period saved by the input capture ISR, and set the compare register by adding the period to the counter’s value and storing that number in the OC1A register.

All written programs should be tested with Simulator before the lab. Remember to use the I/O window to verify hardware output.

In the Lab

You will need to connect an oscilloscope and a function generator to the correct pins on the Arduino board. Have your connections verified by the TA, before powering it up. **Before connecting the function generator to the board, verify that the signal it is producing is in the range 0-5 V – voltages outside of this range will damage the AVR.**

1. Load and run your first program. Connect pin `OC1A` to an oscilloscope. Run the program and verify that the period and duty cycle are correct. Verify them for duty cycles of 12.5%, 25%, 50%, 75%, and 90%. Are the results what you expected? Note the different durations carefully. Can you see any deviation from what you expected? Why? (Or why not?) What are the limits of your program? What are the limits of the high and low portions of your signal? Why?
2. Load and run program two. Ensure that the timing is as designed, and listen to the music. What is the smallest and largest signal frequency that you can generate? Where are the limitations coming from?
3. Load and run program three using both the signal generator and the oscilloscope. Note the results and explain them to the TA.

After the Lab

Clean up your programs to submit to the TA. Answer all the questions appearing above. Calculate the longest period that you can measure with the last program.